

Foundations for programming languages

1: Introduction

(with a simple language)

Isao Sasano

Schedule

- Lectures, Small exams, Mid-term exam, Final exam
- Evaluation
 - Mid-term exam: M point / 40 point
 - Final exam: F point / 50 point
 - Small exam: S point / 10 point
 - The overall score: $S+M+F*(100-(S+M))/50$

Contact information

- Isao Sasano

Office: Toyosu campus 14K32 on the 14th floor

E-mail: sasano@sic.shibaura-it.ac.jp

web: <https://www.cs.ise.shibaura-it.ac.jp/~sasano/index.html>

The lecture page is linked from the above web page.

Machine language

- Machine language is the native language of a computer, directly interpreted and executed by the computer.

The term *code* originally referred to a program written in a machine language.

- von Neumann machine
 - designed in Princeton in 1946
 - Turing machine + random access + IO

Machine language

A program written in some machine language is a sequence of numbers.

[A code fragment for the von Neumann machine]

00000010101111001010

00000010111111001000

00000011001110101000

(This fragment adds the numbers of locations 10 and 11 and stores the result in location 12.)

Modern computers are said to have a von Neumann architecture.

In assembly language instructions are represented by symbols.

Programming languages

Programming languages are expected not to depend on specific machines (i.e., expected to be high level).

Properties that programming languages should have:

- High level
 - Logical structure of programs are concisely described.
- Semantics of programs being strictly defined
 - Semantics (behavior) of all the programs are completely specified.
- Efficient
 - Programs are transformed into efficient code in machine languages.

Benefits of high-level languages

Higher-level languages have taken the place of machine and assembly languages in virtually all areas of programming.

- Readable familiar notations
- machine independence (i.e., portability)
- Availability of program libraries
- Consistency checking that detects syntax and type errors

(ex.) The language C

- UNIX kernel (originally written in assembly languages) was rewritten in the language C in 1973.
- It becomes much easier to modify codes and support new devices
- UNIX was made available on hardware other than PDP-11 without modifying most of the codes.

Classification of programming languages

Programming languages are roughly classified into the following four according to their computational models.

- Imperative languages (or procedural languages)
- Functional languages
- Object-oriented languages
- Logic programming languages

What languages provide (1)

- Computational model (cf. previous page)
- Data types (and operations on them)
 - (ex.) The language C provides primitive data types such as int, char, double, etc. The language C also provides mechanisms to construct large objects using smaller objects, such as arrays, structures (records in general terminology), pointers, and unions. The language C also provides operators to access the components of compound data, such as dot operator (member access operator) for structures (records).

What languages provide (2)

- Abstraction
 - (ex. 1) Variables in imperative languages
 - A variable abstracts some address in memory.
 - Variables are instantiated in compile time and runtime.
 - (ex. 2) Functions in imperative languages
 - A function definition abstracts some computation.
 - A function call (application) instantiates the function body by assigning the actual parameters to the corresponding formal parameters.
 - (ex. 3) goto statement in C, exceptions in Java
 - Jumps are abstracted and instantiated.
 - (ex. 4, advanced) Polymorphic types in functional languages
 - Types are abstracted and instantiated.
- Checking mechanisms
 - (ex.) Syntax checking (parsing), type checking
 - In compile time syntax errors and type errors are detected.

Syntax of programming languages

- (ex) Syntax of the languages of sequence of numbers in BNF notation

$\langle d \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\langle \text{digit_seq} \rangle ::= \langle d \rangle$

$\mid \langle d \rangle \langle \text{digit_seq} \rangle$

$\langle \text{real-number} \rangle ::= \langle \text{digit_seq} \rangle . \langle \text{digit_seq} \rangle$

Usually the grammar of programming languages belong to context-free grammars. BNF notation is a concise way to describe context-free grammars.

Semantics of programming languages

(ex.) Syntax of the language of dates

$\langle \text{date} \rangle ::= \langle \text{d} \rangle \langle \text{d} \rangle / \langle \text{d} \rangle \langle \text{d} \rangle / \langle \text{d} \rangle \langle \text{d} \rangle \langle \text{d} \rangle \langle \text{d} \rangle$

01/02/2001

In US this represents January 2, 2001.

In some other countries is represents February 1, 2001.

A programming language is specified by defining its syntax and semantics.

Definitions and explanations of programming languages

- Tutorial
 - Tutorial of a programming language introduces its outline.
- Reference manual
 - Reference manual of a programming language describes its syntax in BNF and semantics in some natural language (typically English). Syntax is formally defined by BNF and semantics is informally described.
- Formal definition
 - Formal definition of a programming language is a description of syntax (in BNF) and semantics in some formal description such as operational semantics, denotational semantics, and axiomatic semantics which suits formal arguments.

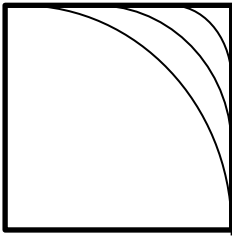
A simple language ---Little Quilt



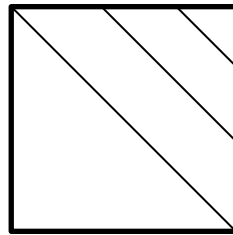
A quilt

Little Quilt language

- Little quilt is a language that makes a quilt that consists of two basic figures.



a



b

Expressions of Little Quilt

$\langle \text{exp} \rangle ::= a$

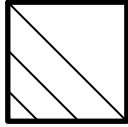
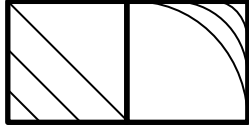
| b

| $\text{turn}(\langle \text{exp} \rangle)$

| $\text{sew}(\langle \text{exp} \rangle, \langle \text{exp} \rangle)$

- $\text{turn}(e)$ --- represents the quilt obtained by rotating 90 degrees to the right the quilt represented by the expression e .
- $\text{sew}(e1, e2)$ --- represents the quilt obtained by sewing the two quilt $e1$ and $e2$ ($e1$ is in the left side and $e2$ is in the right side). The quilt $e1$ and $e2$ must have the same height.

Examples of expressions of Little Quilt

expressions	quilts
b	
turn (b)	
turn (turn (b))	
a	
sew (turn (turn (b))), a)	

Exercise 1

Illustrate the quilt represented by the following expression.

turn (sew (turn (b), turn (b)))

Function declaration

fun unturn (x) = turn (turn (turn (x)))

This function represents the operation of left turn.

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

This function sews the quilt x and y, where x is in the upper side and y is in the lower. The quilt x and y must have the same width.

Function declaration provides a way to name computation patterns that frequently occur.

Syntax of function declaration:

fun <name> (<formals>) = <exp>

<formals> ::= <name> | <name>, <formals>

Later we add <name> and function application to the definition of <exp>.

Local declarations (let expressions)

(ex.)

let

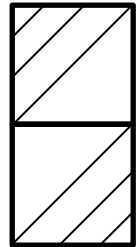
fun unturn (x) = turn (turn (turn (x)))

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

in

pile (unturn (b), turn (b))

end



Syntax of let expressions

let <decls> **in** <exp> **end**

We define <decls> later.

The scope of each of the names of the functions declared in <decls> is in the declarations after its declaration in <decls> and between **in** and **end**, where the scopes of the same name declared there are excluded.

Exercise 2

Illustrate the quilt represented by the following expression.

let

 fun f (x) = turn (turn (x))

in

 f (f (b))

end

Syntax that names some value (i.e., quilt)

(ex.)

let

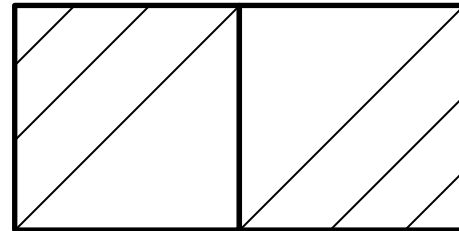
val x = unturn (b)

val y = turn (b)

in

sew (x,y)

end



Syntax for value declarations

val <name> = <exp>

Scopes of names are defined in the same way as function declarations

A larger example

let

fun unturn (x) = turn (turn (turn (x)))

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

val aa = pile (a, turn (turn (a)))

val bb = pile (unturn (b), turn (b))

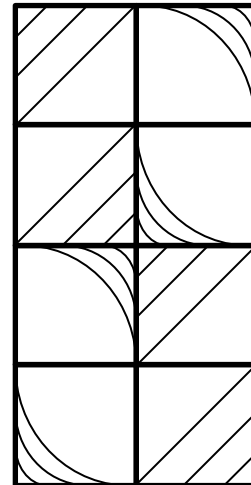
val p = sew (bb, aa)

val q = sew (aa, bb)

in

pile (p,q)

end



Syntax of the Little Quilt language

$\langle \text{exp} \rangle ::= a \mid b \mid \langle \text{name} \rangle \mid \langle \text{name} \rangle (\langle \text{actuals} \rangle)$
 $\mid \text{turn} (\langle \text{exp} \rangle) \mid \text{sew} (\langle \text{exp} \rangle , \langle \text{exp} \rangle)$
 $\mid \textbf{let} \langle \text{decls} \rangle \textbf{in} \langle \text{exp} \rangle \textbf{end}$

$\langle \text{actuals} \rangle ::= \langle \text{exp} \rangle \mid \langle \text{exp} \rangle , \langle \text{actuals} \rangle$

$\langle \text{decls} \rangle ::= \langle \text{decl} \rangle \mid \langle \text{decl} \rangle \langle \text{decls} \rangle$

$\langle \text{decl} \rangle ::= \textbf{fun} \langle \text{name} \rangle (\langle \text{formals} \rangle) = \langle \text{exp} \rangle$
 $\mid \textbf{val} \langle \text{name} \rangle = \langle \text{exp} \rangle$

$\langle \text{formals} \rangle ::= \langle \text{name} \rangle \mid \langle \text{name} \rangle , \langle \text{formals} \rangle$

$\langle \text{name} \rangle$ is strings, usually processed by lexical analysis.

In BNF, $\langle \text{name} \rangle$ can be defined as follows.

$\langle \text{name} \rangle ::= \langle \text{c} \rangle \mid \langle \text{c} \rangle \langle \text{name} \rangle$

$\langle \text{c} \rangle ::= a \mid b \mid c \mid d \mid e \dots$